FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	*Benedict Carl R. Roxas*
Section:	*AN42*
Professor:	*Ms. Crisola G. Tan*
Date:	*Oct 5, 2026*

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) *is an intelligent problem-solving approach used in* **Knowledge Representation and Reasoning (KRR)**. *Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.*

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, *including retrieval, reuse, revision, and retention.*
2. **Represent cases effectively** *using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.*
3. **Implement a similarity assessment method** *to identify relevant past cases based on defined attributes.*
4. **Adapt retrieved solutions** *to meet the needs of a new problem scenario based on the chosen case study.*
5. **Demonstrate learning in CBR** *by saving the newly solved case into the system's case base.*
6. **Develop collaboration skills** *through pair programming and joint problem analysis.*
7. **Apply knowledge representation and reasoning concepts** *to real-world problems in disaster response, legal reasoning, or intelligent tutoring.*

Materials Needed

1. Hardware

- *A laptop or desktop computer*
- *Stable internet connection (for research, documentation, submission)*

2. Software

A programming environment:

- *Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)*
- *Any text editor (Notepad++, Sublime, VS Code) if needed*

3. File Formats & Tools

Word processor for documentation:

- *MS Word, Google Docs, or LibreOffice*
- *Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)*

4. Reference Materials

Lecture notes on:

- *Knowledge Representation & Reasoning*
- *Case-Based Reasoning cycle*
- *Similarity measures*

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)

- *Google Drive for shared documentation*
- *GitHub/GitLab for code collaboration*

General Instructions

1. *This* **Technical Assessment (TA3)** *must be completed by Pair.*
2. *Please ensure that you are grouped as pairs in the People tab.*
3. *Select only one case study from the list below.*
4. *Include clear comments in your code to describe its logic.*
5. *Each program should display the correct input, process, and output.*
6. *Ensure that your script displays clear and readable outputs.*
7. *Compile all Python codes and corresponding outputs into one PDF file.*
8. *Save it as* **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. *Submit the* **python file** *and* **PDF** *file.*

📌 *Step-by-Step Instructions*

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*

3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** *stored as dictionaries, JSON objects, arrays, or any structure.*

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- *Attribute matching*
- *Weighted scoring*
- *Simple rule-based comparison*

Output should show: (sample)

✓ *Similarity score or ranking*

✓ *The most relevant past case retrieved*

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** *Adjust evacuation plans based on population size*
- **Legal DSS:** *Modify arguments due to new digital evidence*
- **ITS:** *Give personalized feedback based on student errors*

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ *"New case saved" or similar confirmation*

✓ *Updated case list*

7. Display clear program outputs

Your output must include:

- *New problem description*
- *Retrieved similar case*
- *Adapted final solution*
- *Updated case base*

8. Submission Requirements

Submit the following:

A. Program File

- *Python*

B. Short Documentation (1–2 pages)

Include:

- *Title of chosen case study*
- *Description of your case base*
- *Explanation of similarity method*
- *Adaptation logic*
- *Screenshots of program execution*

C. Pair Reflection (5–7 sentences)

Discuss:

- *What each partner contributed*
- *What challenges you encountered*
- *What you learned about CBR*

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- *Past cases include typhoons, floods, and earthquakes.*
- *New event: Typhoon with expected flooding in urban areas.*
- *System retrieves similar events: Typhoon Ursula (2019) and Typhoon Odette (2021).*
- *It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.*
- *After execution, the system stores new outcomes for improved future reasoning.*

Learning Outcome:

CBR helps governments respond faster by using experiential knowledge rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- *online theft*
- *digital fraud*
- *unauthorized bank transfers*

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

Chosen Case Study: CS3

```
# CASE-BASED REASONING (CBR)
# Intelligent Tutoring System

# Step 1: Create the case base
case_base = [
    {
        "student": "Student 1",
        "problem": {
            "error": "wrong_formula",
            "topic": "algebra",
            "difficulty": "medium"
        },
        "solution": "Review the correct algebra formula and solve similar
examples."
    },
    {
        "student": "Student 2",
        "problem": {
            "error": "calculation_error",
            "topic": "algebra",
            "difficulty": "easy"
        },
        "solution": "Practice basic calculations and check each step."
    },
    {
        "student": "Student 3",
        "problem": {
            "error": "wrong_formula",
            "topic": "geometry",
            "difficulty": "medium"
```

```
        },
        "solution": "Review geometry formulas and practice identifying the
correct formula."
    },
    {
        "student": "Student 4",
        "problem": {
            "error": "concept_error",
            "topic": "algebra",
            "difficulty": "hard"
        },
        "solution": "Review the basic algebra concept and work through guided
examples."
    }
]


# Step 2: Get a new student problem

print("INTELLIGENT TUTORING SYSTEM")
print("\nEnter the new student's problem:")

error = input("Error type (wrong_formula/calculation_error/concept_error):
").lower()
topic = input("Topic (algebra/geometry): ").lower()
difficulty = input("Difficulty (easy/medium/hard): ").lower()

new_problem = {
    "error": error,
    "topic": topic,
    "difficulty": difficulty
}


# Step 3: Similarity Assessment

def calculate_similarity(new_problem, old_problem):
    score = 0

    if new_problem["error"] == old_problem["error"]:
        score += 2
```

```
    if new_problem["topic"] == old_problem["topic"]:
        score += 2

    if new_problem["difficulty"] == old_problem["difficulty"]:
        score += 1

    return score


# Step 4: Retrieve the most similar case

best_case = None
best_score = -1

print("\nSimilarity Scores:")

for case in case_base:
    score = calculate_similarity(new_problem, case["problem"])

    print(case["student"], ":", score, "/ 5")

    if score > best_score:
        best_score = score
        best_case = case

print("\nRetrieved Case:")
print(best_case["student"])
print("Similarity Score:", best_score, "/ 5")
print("Previous Solution:", best_case["solution"])


# Step 5: Reuse and Adapt the Solution

adapted_solution = best_case["solution"]

if error == "wrong_formula":
    adapted_solution = "Review the correct formula for " + topic + " and solve
similar practice problems."
```

```
elif error == "calculation_error":
    adapted_solution = "Practice " + topic + " calculations and check each step carefully."

elif error == "concept_error":
    adapted_solution = "Review the basic concepts of " + topic + " and complete guided practice problems."


print("\nAdapted Final Solution:")
print(adapted_solution)


# Step 6: Retain the New Case

new_case = {
    "student": "New Student",
    "problem": new_problem,
    "solution": adapted_solution
}

case_base.append(new_case)

print("\nNew case saved successfully!")


# Step 7: Display Updated Case Base

print("\nUpdated Case Base:")

for i, case in enumerate(case_base, 1):
    print(
        i,
        "-",
        case["student"],
        "|",
        case["problem"],
        "|",
        case["solution"]
    )
```

```
INTELLIGENT TUTORING SYSTEM

Enter the new student's problem:
Error type (wrong_formula/calculation_error/concept_error): wrong_formula
Topic (algebra/geometry): algebra
Difficulty (easy/medium/hard): medium

Similarity Scores:
Student 1 : 5 / 5
Student 2 : 2 / 5
Student 3 : 3 / 5
Student 4 : 2 / 5

Retrieved Case:
Student 1
Similarity Score: 5 / 5
Previous Solution: Review the correct algebra formula and solve similar examples.

Adapted Final Solution:
Review the correct formula for algebra and solve similar practice problems.

New case saved successfully!

Updated Case Base:
1 - Student 1 | {'error': 'wrong_formula', 'topic': 'algebra', 'difficulty': 'medium'} | Review the correct algebra formula and solve similar examples.
2 - Student 2 | {'error': 'calculation_error', 'topic': 'algebra', 'difficulty': 'easy'} | Practice basic calculations and check each step.
3 - Student 3 | {'error': 'wrong_formula', 'topic': 'geometry', 'difficulty': 'medium'} | Review geometry formulas and practice identifying the correct formula.
4 - Student 4 | {'error': 'concept_error', 'topic': 'algebra', 'difficulty': 'hard'} | Review the basic algebra concept and work through guided examples.
5 - New Student | {'error': 'wrong_formula', 'topic': 'algebra', 'difficulty': 'medium'} | Review the correct formula for algebra and solve similar practice problems.
```

Documentation:

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Intelligent Tutoring System (ITS)

Chosen Case Study

For this activity, I chose the Intelligent Tutoring System (ITS). The program helps students by looking at problems from previous students and using their solutions as a guide for a new problem.

Case Base

The program contains four previous student cases. Each case has an error type, topic, difficulty level, and a suggested solution. The cases are stored using Python dictionaries. These cases serve as the information that the program uses when looking for a solution to a new student's problem.

Similarity Method

The program checks how similar the new problem is to each case. It looks at three things: the type of error, the topic, and the difficulty. A matching error gives 2 points, a matching topic gives 2 points, and a matching difficulty gives 1 point. The case with the most points is considered the closest match.

Adaptation

After finding the closest case, the program creates a new solution based on the student's problem. The solution changes depending on the error type and topic. For example, a student who has a wrong formula in algebra will be told to review the correct formula and practice similar problems.

Retaining the New Case

The program saves the new student's problem and solution after it finishes. This new case is added to the existing case base. The program then displays the updated list of cases. This shows how CBR can keep new information and use it later.

Program Output

The program first asks for the student's error type, topic, and difficulty. It then shows the similarity scores for the previous cases and identifies the closest case. After that, it displays the adapted solution and confirms that the new case has been saved.

Reflection

I made a Case-Based Reasoning program that works like a simple tutoring system. I learned how to store previous problems and their solutions using Python dictionaries. One part that I found difficult was creating the similarity score because the program needs to compare different parts of each case. I also had to make sure that the solution given by the program matched the student's problem. Testing the program helped me find and fix mistakes in the code. Through this activity, I understood how CBR can use past experiences to help with new problems. I also learned that saving new cases can make the system more useful when solving similar problems in the future.